

Tests E2E avec Playwright

Jour 2 — Architecture, Locators, POM & Debug

Formation Frontend Testing · Angular Ecosystem



Programme — Jour 2

01 Introduction à Playwright

02 Architecture de Playwright

03 Installation & configuration

04 playwright.config.ts en détail

05 Écrire des tests Playwright

06 Locators & sélecteurs modernes

07 Interagir avec le DOM

08 Page Object Model (POM)

09 Débugger les tests

10 Playwright & Angular

11 CI/CD & Bonnes pratiques

12 Migration depuis Protractor

| 01 · Introduction à Playwright

Le standard des tests e2e



Qu'est-ce que Playwright ?

- Framework d'automatisation de navigateurs par **Microsoft**
- Version actuelle : 1.48+
- Supporte : **Chromium**, **Firefox**, **WebKit** (Safari)
- Langages : **TypeScript**, **JavaScript**, **Python**, **Java**, **C#**
- Inclut son propre test runner (`@playwright/test`)

*Playwright est le **successeur officiel** de Protractor pour les tests Angular e2e*

VS Playwright vs les alternatives

Critère	Playwright	Cypress	Selenium
Multi-navigateurs	✓ Natif	⚠ Limité	✓ Natif
Multi-onglets	✓	✗	✓
API moderne	✓ async/await	✓ chaînable	✗
Vitesse	🚀 Très rapide	⚡ Rapide	🐢 Lent
Mobile	✓ Émulation	⚠	⚠
Prix	Gratuit	Gratuit/Payant	Gratuit

☀ Les forces de Playwright

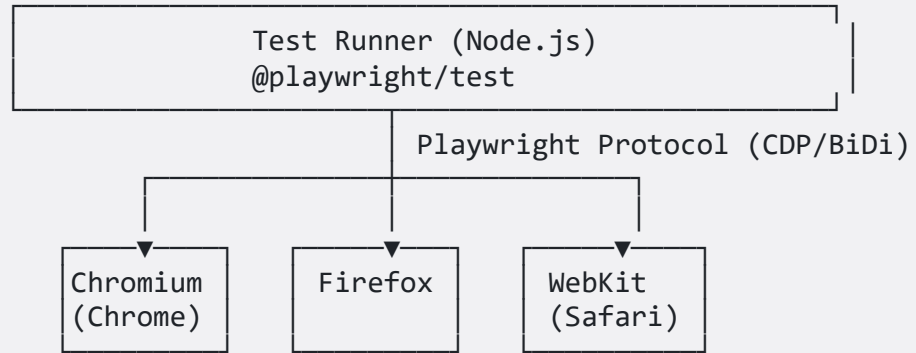
- **Auto-waiting** : attend automatiquement que les éléments soient prêts
- **Isolation** : chaque test dans son propre contexte navigateur
- **Trace Viewer** : enregistrement complet de l'exécution des tests
- **UI Mode** : interface visuelle pour déboguer interactivement
- **Locators robustes** : `getByRole` , `getByLabel` centrés sur l'accessibilité
- **Fixtures** : système de setup/teardown puissant et composable

| 02 · Architecture de Playwright

Comment Playwright communique avec les navigateurs



Architecture générale





Protocoles navigateur

- Chrome DevTools Protocol (CDP) pour Chromium
- Firefox Remote Protocol pour Firefox
- WebKit Inspector Protocol pour WebKit/Safari
- Playwright abstrait ces différences → API unifiée
- Contrairement à Selenium : pas besoin de WebDriver séparé
- Les navigateurs sont embarqués avec Playwright

Les concepts fondamentaux

Browser

Instance d'un navigateur (Chromium, Firefox, WebKit)

BrowserContext

Contexte isolé = profil navigateur (cookies, localStorage)

Page

Un onglet navigateur. Point d'entrée principal des tests.

Locator

Référence lazy vers un élément DOM. Base de l'interaction.



Cycle de vie d'un test Playwright

```
// Playwright gère automatiquement le cycle de vie
test('user can login', async ({ page }) => {
  // 1. page = un nouvel onglet dans un BrowserContext isolé
  await page.goto('https://myapp.com/login');

  // 2. Interaction avec les éléments
  await page.getByLabel('Email').fill('user@example.com');
  await page.getByLabel('Password').fill('secret');
  await page.getByRole('button', { name: 'Login' }).click();

  // 3. Assertions
  await expect(page).toHaveURL('/dashboard');
  // 4. Playwright ferme le contexte automatiquement
});
```

⚡ Auto-waiting — La magie de Playwright

```
// ❌ Avec Selenium : gérer manuellement l'attente
driver.wait(until.elementLocated(By.id('btn')), 5000);
driver.findElement(By.id('btn')).click();

// ✅ Avec Playwright : l'attente est automatique
await page.getByRole('button', { name: 'Submit' }).click();
// Playwright attend que le bouton soit :
// • Visible
// • Stable (pas en train de bouger)
// • Activé (pas disabled)
// • Recevant les événements
```

03 · Installation & Configuration

Mise en place de Playwright dans un projet Angular



Installation dans un projet Angular

```
# Méthode 1 : avec le schématic Angular (recommandé)  
ng add @playwright/test
```

```
# Méthode 2 : installation manuelle  
npm init playwright@latest
```

```
# Répondre aux questions :  
# ✓ Where to put your end-to-end tests? → e2e/  
# ✓ Add a GitHub Actions workflow? → Yes  
# ✓ Install Playwright browsers? → Yes
```



Installation — Ce qui est créé

```
mon-projet/
├── e2e/
│   ├── example.spec.ts    ← test d'exemple
│   └── pages/             ← Page Objects
├── playwright.config.ts   ← configuration principale
├── .github/
│   └── workflows/
│       └── playwright.yml ← workflow CI
└── package.json           ← scripts ajoutés
```



Installation des navigateurs

```
# Installer tous les navigateurs Playwright  
npx playwright install
```

```
# Installer seulement Chromium  
npx playwright install chromium
```

```
# Installer + dépendances système (Linux/CI)  
npx playwright install --with-deps chromium
```

```
# Vérifier l'installation  
npx playwright --version
```




Lancer les tests

```
# Lancer tous les tests (headless)
npx playwright test

# Avec interface graphique (UI Mode) ← recommandé en dev
npx playwright test --ui

# Tests en mode headed (navigateur visible)
npx playwright test --headed

# Un fichier spécifique
npx playwright test e2e/login.spec.ts

# Avec un navigateur spécifique
npx playwright test --project=firefox
```

| 04 · playwright.config.ts en détail

Maîtriser la configuration Playwright

playwright.config.ts — Structure

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './e2e',           // dossier des tests
  fullyParallel: true,        // parallélisation maximale
  forbidOnly: !!process.env.CI, // interdit .only en CI
  retries: process.env.CI ? 2 : 0, // retries en CI
  workers: process.env.CI ? 1 : undefined,
  reporter: 'html',           // rapport HTML
  use: {
    baseURL: 'http://localhost:4200',
    trace: 'on-first-retry', // trace sur échec
  },
  // ...
});
```



playwright.config.ts — Projets

```
projects: [  
  {  
    name: 'chromium',  
    use: { ...devices['Desktop Chrome'] },  
  },  
  {  
    name: 'firefox',  
    use: { ...devices['Desktop Firefox'] },  
  },  
  {  
    name: 'webkit',  
    use: { ...devices['Desktop Safari'] },  
  },  
  // Test mobile  
  {  
    name: 'Mobile Chrome',  
    use: { ...devices['Pixel 5'] },  
  },  
],
```



playwright.config.ts — Serveur Angular

```
// Démarrer automatiquement Angular avant les tests
webServer: {
  command: 'ng serve',
  url: 'http://localhost:4200',
  reuseExistingServer: !process.env.CI,
  timeout: 120 * 1000, // 2 minutes max pour démarrer
},

// Configuration globale use{}
use: {
  baseURL: 'http://localhost:4200',
  screenshot: 'only-on-failure',
  video: 'retain-on-failure',
  trace: 'on-first-retry',
},
```

⚙️ Options importantes de `use{}`

Option	Valeurs	Description
<code>baseUrl</code>	URL	URL de base pour <code>goto('/')</code>
<code>headless</code>	bool	Mode sans interface
<code>viewport</code>	<code>{w, h}</code>	Taille de la fenêtre
<code>screenshot</code>	off/on/only-on-failure	Captures d'écran
<code>video</code>	off/on/retain-on-failure	Enregistrement vidéo
<code>trace</code>	off/on/on-first-retry	Trace pour debug
<code>locale</code>	<code>'fr-FR'</code>	Langue du navigateur
<code>geolocation</code>	<code>{lat, lng}</code>	Position GPS simulée

| 05 · Écrire des Tests Playwright

La syntaxe et les patterns fondamentaux



Structure d'un test Playwright

```
import { test, expect } from '@playwright/test';

// describe : groupe de tests (optionnel)
test.describe('Login Feature', () => {

  // hook : avant chaque test
  test.beforeEach(async ({ page }) => {
    await page.goto('/login');
  });

  // test individuel
  test('should login with valid credentials', async ({ page }) => {
    await page.getByLabel('Email').fill('user@example.com');
    await page.getByLabel('Password').fill('secret123');
    await page.getByRole('button', { name: 'Se connecter' }).click();
    await expect(page).toHaveURL('/dashboard');
  });
});
```




Hooks disponibles

```
test.beforeAll(async ({ browser }) => {
  // Exécuté une fois avant tous les tests du fichier
  // Utile pour créer des données de test
});

test.beforeEach(async ({ page }) => {
  // Avant chaque test - navigation, login...
  await page.goto('/');
});

test.afterEach(async ({ page }, testInfo) => {
  // Après chaque test - nettoyage
  if (testInfo.status !== testInfo.expectedStatus) {
    await page.screenshot({ path: `failure-${testInfo.title}.png` });
  }
});

test.afterAll(async () => {
  // Nettoyage final - supprimer données de test
});
```



Focus, Skip & Annotations

```
// Focus sur un test spécifique
test.only('only this test runs', async ({ page }) => { });

// Passer un test
test.skip('not yet implemented', async ({ page }) => { });

// Marquer comme attendu échouer
test.fail('known bug #123', async ({ page }) => { });

// Skip selon une condition
test.skip(process.env.CI === 'true', 'Skip in CI');

// Annotations pour le rapport
test('user login @smoke @regression', async ({ page }) => { });
// Filtrer : npx playwright test --grep @smoke
```



Les Assertions Playwright

```
// Assertions sur la page
await expect(page).toHaveURL('/dashboard');
await expect(page).toHaveTitle('Mon Application');

// Assertions sur les locators
const heading = page.getByRole('heading', { name: 'Bienvenue' });
await expect(heading).toBeVisible();
await expect(heading).toHaveText('Bienvenue Alice');

// Assertions sur les inputs
const input = page.getByLabel('Email');
await expect(input).toHaveValue('user@example.com');
await expect(input).toBeFocused();
await expect(input).toBeEnabled();
```



Assertions — Soft Assertions

```
// Par défaut : une assertion échouée stoppe le test
await expect(locator).toHaveText('Expected');

// Soft assertion : continue après l'échec et reporte à la fin
await expect.soft(locator).toHaveText('Expected Text');
await expect.soft(page).toHaveURL('/dashboard');
// Le test continue même si les assertions précédentes échouent
// Toutes les erreurs sont reportées à la fin du test
```

Les Fixtures Playwright

```
// Fixtures intégrées disponibles dans chaque test
test('example', async ({
  page,           // Page – onglet isolé
  browser,        // Browser – instance navigateur
  browserName,    // string – 'chromium' | 'firefox' | 'webkit'
  context,        // BrowserContext – profil navigateur
  request,        // APIRequestContext – pour les API calls
  baseURL,        // string – URL de base configurée
}) => {
  // ...
});
```

06 · Locators & Sélecteurs Modernes

La philosophie "find by user-facing attributes"

La philosophie des Locators

- **Locator** = référence *lazy* vers un ou plusieurs éléments
- Il ne cherche pas l'élément au moment de sa création
- Il cherche **au moment de l'interaction** (auto-retry)
- **Priorité : utiliser des sélecteurs accessibles et stables**

« Find elements the way users see them, not the way developers write them »

Locators — getByRole (recommandé ★)

```
// Boutons
await page.getByRole('button', { name: 'Valider' }).click();
await page.getByRole('button', { name: /valider/i }).click(); // regex

// Liens
await page.getByRole('link', { name: 'Accueil' }).click();

// Checkboxes et radios
await page.getByRole('checkbox', { name: 'Accepter CGU' }).check();
await page.getByRole('radio', { name: 'Mensuel' }).click();

// Tableaux
const table = page.getByRole('table', { name: 'Utilisateurs' });

// Headings
await expect(page.getByRole('heading', { level: 1 }))
  .toHaveText('Mon Titre');
```




Locators — getByLabel & getByPlaceholder

```
// Cibler un input par son label associé
const emailInput = page.getByLabel('Adresse email');
await emailInput.fill('alice@example.com');

// Cibler par placeholder
const searchInput = page.getByPlaceholder('Rechercher...');
await searchInput.fill('Angular');

// Combinaison : label + action
await page.getByLabel('Mot de passe').fill('secret123');
await page.getByLabel('Se souvenir de moi').check();
```

Locators — getByText & getByAltText

```
// Texte exact ou partiel
await page.getByText('Bienvenue').click();
await page.getByText(/bienvenue/i).click(); // insensible à la casse

// Texte exact uniquement
await page.getByText('Connexion', { exact: true }).click();

// Images par alt text
await page.getByAltText('Logo de la société').click();

// Éléments par title
await page.getByTitle('Fermer la fenêtre').click();
```

Locators — getByTestId

```
// Via data-testid (attribut custom) – dernier recours
await page.getByTestId('submit-button').click();

// HTML correspondant :
// <button data-testid="submit-button">Envoyer</button>

// Configurer l'attribut custom dans playwright.config.ts
use: {
  testIdAttribute: 'data-cy', // compatibilité Cypress
}
```



Locators — CSS & XPath (à éviter)

```
// CSS — moins robuste, à éviter si possible
await page.locator('.btn-primary').click();
await page.locator('#submit').click();
await page.locator('[data-id="user-42"]').click();

// XPath — encore moins recommandé
await page.locator('xpath=//button[@type="submit"]').click();

// Combinaison de locators (filtering)
await page.locator('.user-card')
  .filter({ hasText: 'Alice' })
  .getByRole('button', { name: 'Modifier' })
  .click();
```

Locators — Chaînage et filtrage

```
// Trouver un élément dans un contexte
const userCard = page.locator('.user-card').filter({ hasText: 'Alice' });
await userCard.getByRole('button', { name: 'Modifier' }).click();

// nth() – cibler par index
await page.getByRole('listitem').nth(0).click(); // premier élément

// first() / last()
await page.getByRole('row').first().click();
await page.getByRole('row').last().click();

// Count
const count = await page.getByRole('listitem').count();
expect(count).toBe(5);
```

Les ElementFinder → Locators (migration)

Protractor	Playwright
<code>element(by.id('x'))</code>	<code>page.locator('#x')</code>
<code>element(by.css('.btn'))</code>	<code>page.locator('.btn')</code>
<code>element(by.buttonText('OK'))</code>	<code>page.getByRole('button', {name:'OK'})</code>
<code>element(by.model('name'))</code>	<code>page.getByLabel('Name')</code>
<code>element.all(by.css('.item'))</code>	<code>page.locator('.item')</code>
<code>\$\$('.item')</code>	<code>page.locator('.item')</code>

Atelier 5 — Premiers tests Playwright

 Référence : *exercices-jour2.md* — Exercice 1

Atelier 5 — Objectifs

- Installer Playwright dans le projet Angular du Jour 1
- Écrire un premier test de navigation
- Utiliser `getByRole`, `getByLabel`, `getByText`
- Vérifier avec `toHaveURL`, `toBeVisible`, `toHaveText`
- Lancer en mode headed et observer l'exécution

Durée estimée : 30 minutes

Référence : `exercices-jour2.md` → Exercice 1

| 07 · Interagir avec le DOM

Les actions disponibles dans Playwright



Actions — Clics

```
// Clic simple
await page.getByRole('button', { name: 'Valider' }).click();

// Double-clic
await page.getByText('Fichier').dblclick();

// Clic droit
await page.getByText('Élément').click({ button: 'right' });

// Clic avec modificateur
await page.getByText('Item').click({ modifiers: ['Shift'] });
await page.getByText('Item').click({ modifiers: ['Control'] });

// Forcer le clic (contourne l'auto-waiting)
await page.locator('.hidden-btn').click({ force: true });
```



Actions — Saisie de texte

```
// fill() : remplace tout le contenu (recommandé)
await page.getByLabel('Prénom').fill('Alice');

// type() : simule la frappe caractère par caractère
await page.getByLabel('Commentaire').pressSequentially('Bonjour');

// clear() : vider un champ
await page.getByLabel('Recherche').clear();

// press() : touches clavier
await page.getByLabel('Recherche').press('Enter');
await page.getByLabel('Code').press('Control+A');
await page.getByLabel('Code').press('Backspace');
```



Actions — Formulaires

```
// Sélection dans un <select>
await page.getByLabel('Pays').selectOption('FR');
await page.getByLabel('Pays').selectOption({ label: 'France' });
await page.getByLabel('Langues').selectOption(['fr', 'en']); // multi

// Checkbox
await page.getByLabel('Accepter CGU').check();
await page.getByLabel('Newsletter').uncheck();
await page.getByLabel('Option').setChecked(true);

// Upload de fichier
await page.getByLabel('Avatar').setInputFiles('path/to/file.jpg');
await page.getByLabel('Documents').setInputFiles([
  'file1.pdf', 'file2.pdf'
]);
```



Actions — Hover, Focus, Drag

```
// Hover (survol)
await page.getByText('Menu').hover();

// Focus
await page.getByLabel('Email').focus();

// Drag and drop
await page.getByText('Source').dragTo(page.getByText('Cible'));

// Drag avec coordonnées
await page.getByText('Draggable').dragTo(
  page.getByText('Drop Zone'),
  { targetPosition: { x: 50, y: 50 } }
);

// Scroll vers un élément
await page.getByText('Footer').scrollIntoViewIfNeeded();
```



Screenshots & Assertions visuelles

```
// Screenshot d'un élément
const chart = page.getByTestId('revenue-chart');
await expect(chart).toHaveScreenshot('revenue-chart.png');

// Screenshot de la page entière
await expect(page).toHaveScreenshot('full-page.png', {
  fullPage: true,
  threshold: 0.1, // tolérance 10%
});

// Mettre à jour les screenshots de référence
// npx playwright test --update-snapshots
```

Attentes & Timeouts

```
// Attendre qu'un élément soit visible
await page.getByRole('alert').waitFor({ state: 'visible' });

// Attendre qu'il disparaisse
await page.getByRole('progressbar').waitFor({ state: 'hidden' });

// Attendre un sélecteur
await page.waitForSelector('.loaded');

// Attendre une URL
await page.waitForURL('/dashboard');

// Attendre une réponse réseau
await page.waitForResponse('/api/users');

// Timeout personnalisé
await page.getByText('Slow').waitFor({ timeout: 30000 });
```



Intercepter le réseau

```
// Intercepter et modifier une requête
await page.route('/api/users', async route => {
  await route.fulfill({
    status: 200,
    contentType: 'application/json',
    body: JSON.stringify([ { id: 1, name: 'Alice' } ])
  });
});

// Bloquer des ressources (images, analytics)
await page.route('**/*.{png,jpg,jpeg}', route => route.abort());
await page.route('**/analytics/**', route => route.abort());

// Modifier les headers
await page.route('/api/**', async route => {
  await route.continue({
    headers: { ...route.request().headers(), 'X-Test': 'true' }
  });
});
```




Surveiller les requêtes réseau

```
// Capturer une requête spécifique
const responsePromise = page.waitForResponse('/api/users');
await page.getByRole('button', { name: 'Charger' }).click();
const response = await responsePromise;
expect(response.status()).toBe(200);

// Écouter toutes les requêtes
page.on('request', request => {
  console.log(request.method(), request.url());
});

// Vérifier qu'une requête a été faite
page.on('response', response => {
  expect(response.url()).toContain('/api/users');
});
```

Atelier 6 — Interactions & Réseau

 Référence : *exercices-jour2.md* — Exercice 2

Atelier 6 — Objectifs

- Tester un formulaire de création d'utilisateur
- Utiliser `fill`, `selectOption`, `check`, `press`
- Intercepter les appels API avec `page.route()`
- Simuler une réponse serveur mockée
- Tester les cas d'erreur (500, 404)

Durée estimée : 35 minutes

Référence : `exercices-jour2.md` → Exercice 2

| 08 · Page Object Model (POM)

Organiser ses tests pour la maintenabilité



Pourquoi le Page Object Model ?

- Sans POM : les sélecteurs sont dupliqués partout
- Un changement d'UI → modifier 20 fichiers de test
- Avec POM : chaque page a sa propre classe
- Les sélecteurs sont **centralisés** dans la classe
- Les tests lisent comme des **scénarios** utilisateur

POM = séparation entre localiser les éléments et tester les comportements



Structure d'un projet POM

```
e2e/
├── pages/
│   ├── login.page.ts      ← Page Object Login
│   ├── dashboard.page.ts  ← Page Object Dashboard
│   ├── user-list.page.ts  ← Page Object Liste Utilisateurs
│   └── user-form.page.ts  ← Page Object Formulaire
├── fixtures/
│   └── app.fixture.ts      ← Fixtures partagées
├── tests/
│   ├── auth.spec.ts
│   ├── users.spec.ts
│   └── dashboard.spec.ts
└── playwright.config.ts
```



Page Object — Exemple LoginPage

```
import { type Locator, type Page } from '@playwright/test';

export class LoginPage {
  readonly page: Page;
  readonly emailInput: Locator;
  readonly passwordInput: Locator;
  readonly submitButton: Locator;
  readonly errorMessage: Locator;

  constructor(page: Page) {
    this.page = page;
    this.emailInput = page.getByLabel('Adresse email');
    this.passwordInput = page.getByLabel('Mot de passe');
    this.submitButton = page.getByRole('button', { name: 'Se connecter' });
    this.errorMessage = page.getByRole('alert');
  }
}
```



Page Object — Méthodes d'action

```
// Suite de LoginPage

async goto() {
  await this.page.goto('/login');
}

async login(email: string, password: string) {
  await this.emailInput.fill(email);
  await this.passwordInput.fill(password);
  await this.submitButton.click();
}

async loginAsAdmin() {
  await this.login('admin@example.com', 'admin123');
}

async getErrorMessage(): Promise<string> {
  return await this.errorMessage.textContent() ?? '';
}
}
```




Page Object — Utilisation dans un test

```
import { test, expect } from '@playwright/test';
import { LoginPage } from '../pages/login.page';

test.describe('Authentication', () => {
  let loginPage: LoginPage;

  test.beforeEach(async ({ page }) => {
    loginPage = new LoginPage(page);
    await loginPage.goto();
  });

  test('should login with valid credentials', async () => {
    await loginPage.login('user@example.com', 'password');
    await expect(loginPage.page).toHaveURL('/dashboard');
  });

  test('should show error on invalid password', async () => {
    await loginPage.login('user@example.com', 'wrong');
    expect(await loginPage.getErrorMessage()).toContain('Identifiants incorrects');
  });
});
```



Page Object — Composition

```
export class DashboardPage {
  readonly page: Page;
  readonly welcomeHeading: Locator;
  readonly userMenu: UserMenuComponent; // composant réutilisable
  readonly sidebar: SidebarComponent;

  constructor(page: Page) {
    this.page = page;
    this.welcomeHeading = page.getByRole('heading', { level: 1 });
    this.userMenu = new UserMenuComponent(page);
    this.sidebar = new SidebarComponent(page);
  }

  async navigateTo(section: 'users' | 'settings' | 'reports') {
    await this.sidebar.clickItem(section);
  }
}
```



Fixtures Playwright — Au-delà du POM

```
// fixtures/app.fixture.ts
import { test as base } from '@playwright/test';
import { LoginPage } from '../pages/login.page';
import { DashboardPage } from '../pages/dashboard.page';

type AppFixtures = {
  loginPage: LoginPage;
  dashboardPage: DashboardPage;
  authenticatedPage: DashboardPage; // déjà connecté
};

export const test = base.extend<AppFixtures>({
  loginPage: async ({ page }, use) => {
    await use(new LoginPage(page));
  },
  authenticatedPage: async ({ page }, use) => {
    const loginPage = new LoginPage(page);
    await loginPage.goto();
    await loginPage.loginAsAdmin();
    await use(new DashboardPage(page));
  },
});
```



Fixtures — Utilisation dans les tests

```
import { test, expect } from '../fixtures/app.fixture';

// La fixture 'authenticatedPage' est injectée automatiquement
test('admin can see all users', async ({ authenticatedPage }) => {
  await authenticatedPage.navigateTo('users');

  const userList = authenticatedPage.page.getByRole('list');
  await expect(userList).toBeVisible();
});

// Pas besoin de login dans chaque test !
test('admin can access settings', async ({ authenticatedPage }) => {
  await authenticatedPage.navigateTo('settings');
  await expect(authenticatedPage.page).toHaveURL('/settings');
});
```

Atelier 7 — Page Object Model

 Référence : *exercices-jour2.md* — Exercice 3

Atelier 7 — Objectifs

- Créer un `LoginPage` et un `UserListPage`
- Extraire tous les sélecteurs dans les Page Objects
- Créer une fixture `authenticatedPage`
- Réécrire les tests précédents en utilisant le POM
- Mesurer la réduction de duplication de code

Durée estimée : 40 minutes

Référence : `exercices-jour2.md` → Exercice 3

| 09 · Débugger les Tests Playwright

UI Mode, Inspector & Trace Viewer

Les outils de debug Playwright

UI Mode

Interface visuelle complète avec timeline, logs et locator picker

Playwright Inspector

Débugueur interactif step-by-step avec hover sur les éléments

Trace Viewer

Replay complet d'un test avec screenshots, réseau et console

Screenshots & Vidéos

Captures automatiques sur échec pour analyse a posteriori



UI Mode — Votre meilleur ami

```
# Lancer en UI Mode
npx playwright test --ui

# Fonctionnalités :
# ✓ Voir tous les tests organisés par fichier
# ✓ Lancer un test individuel en un clic
# ✓ Timeline interactive avec chaque action
# ✓ Locator Picker : pointer sur un élément → obtenir le locator
# ✓ Watch Mode : relancer automatiquement les tests modifiés
# ✓ Filtrer par tag, navigateur, statut
```



Playwright Inspector

```
# Ouvrir l'Inspector (pause au début)
npx playwright test --debug

# Dans un test : pause à un point précis
test('debug this', async ({ page }) => {
  await page.goto('/login');
  await page.pause(); // ← pause ici, Inspector s'ouvre
  await page.getByLabel('Email').fill('test@example.com');
});

# Depuis le code
PWDEBUG=1 npx playwright test
```



Trace Viewer — Analyser les échecs

```
// playwright.config.ts
use: {
  trace: 'on-first-retry', // trace à la 1ère réessai
  // ou : 'on'    = toujours
  // ou : 'off'   = jamais
  // ou : 'retain-on-failure' = garder seulement sur échec
}
```

```
# Voir la trace après un test échoué
npx playwright show-report
```

```
# Ou ouvrir directement une trace
npx playwright show-trace trace.zip
```

Trace Viewer — Ce qu'il contient

- **Timeline** : chronologie de toutes les actions
- **Actions** : chaque `click`, `fill`, `goto` avec sa durée
- **Before/After** : screenshot avant et après chaque action
- **Network** : toutes les requêtes HTTP avec détails
- **Console** : logs console du navigateur
- **Source** : le code source du test correspondant

Un outil indispensable pour comprendre pourquoi un test échoue en CI



Générer des locators automatiquement

```
# Mode codegen : enregistrer les interactions
npx playwright codegen http://localhost:4200

# Playwright ouvre :
# 1. Un navigateur pilotable
# 2. Un éditeur de code qui génère le test en temps réel
# Chaque clic génère automatiquement le meilleur locator

# Avec options
npx playwright codegen --viewport-size=1280,720 http://localhost:4200
npx playwright codegen --device="iPhone 13" http://localhost:4200
```



Techniques de debug avancées

```
// Ralentir l'exécution pour observer
// playwright.config.ts
use: {
  launchOptions: {
    slowMo: 500, // 500ms entre chaque action
  }
}

// Capturer un screenshot à un moment précis
await page.screenshot({ path: 'debug.png' });

// Inspecter le HTML de la page
const html = await page.content();
console.log(html);

// Évaluer du JavaScript dans le navigateur
const title = await page.evaluate(() => document.title);
```

| 10 · Playwright & Angular

Tester une vraie application Angular

A Spécificités Angular en e2e

- Attendre que **Zone.js** ait terminé ses opérations
- Gérer le **routing Angular** (`router-outlet`)
- Tester les **guards** d'authentification
- Gérer les **animations Angular** (Material, etc.)

```
// playwright.config.ts – désactiver les animations
use: {
  actionTimeout: 10000,
  // Désactiver les animations CSS pour des tests plus rapides
  contextOptions: {
    reducedMotion: 'reduce',
  }
}
```


A Tester le routing Angular

```
test('should navigate to user details', async ({ page }) => {  
  await page.goto('/users');  
  
  // Cliquer sur un utilisateur  
  await page.getByText('Alice Martin').click();  
  
  // Vérifier la navigation Angular  
  await expect(page).toHaveURL(/\/users\/\d+\/);  
  
  // Vérifier le contenu chargé via le router  
  await expect(page.getByRole('heading', { level: 1 }))  
    .toHaveText('Alice Martin');  
});
```

A Tester les Guards Angular

```
test('should redirect unauthenticated user', async ({ page }) => {
  // Accéder à une route protégée sans être connecté
  await page.goto('/dashboard');

  // Le guard redirige vers login
  await expect(page).toHaveURL('/login');
  await expect(page.getByText('Connexion requise')).toBeVisible();
});

test('should allow authenticated user', async ({ page }) => {
  // Injecter un token dans localStorage avant le chargement
  await page.addInitScript(() => {
    localStorage.setItem('token', 'valid-jwt-token');
  });
  await page.goto('/dashboard');
  await expect(page).toHaveURL('/dashboard');
});
```

A Tester les formulaires réactifs Angular

```
test('should validate reactive form', async ({ page }) => {
  await page.goto('/users/new');

  // Tenter de soumettre sans remplir
  await page.getByRole('button', { name: 'Créer' }).click();

  // Vérifier les messages d'erreur Angular
  await expect(page.getByText('Le prénom est requis')).toBeVisible();
  await expect(page.getByText('Email invalide')).toBeVisible();

  // Remplir et re-soumettre
  await page.getByLabel('Prénom').fill('Alice');
  await page.getByLabel('Email').fill('alice@example.com');
  await page.getByRole('button', { name: 'Créer' }).click();

  await expect(page).toHaveURL('/users');
});
```

A Tester les composants Angular Material

```
test('should interact with mat-select', async ({ page }) => {
  await page.goto('/settings');

  // Ouvrir le mat-select
  await page.getByLabel('Langue').click();

  // Choisir une option dans l'overlay
  await page.getByRole('option', { name: 'Français' }).click();

  // Vérifier la sélection
  await expect(page.getByLabel('Langue')).toContainText('Français');
});

test('should use mat-datepicker', async ({ page }) => {
  const dateInput = page.getByLabel('Date de naissance');
  await dateInput.fill('15/01/1990');
  await page.keyboard.press('Escape'); // fermer le datepicker
});
```

Atelier 8 — Tests Angular Complets

 Référence : *exercices-jour2.md* — Exercice 4

Atelier 8 — Objectifs

- Tester le flux complet : login → dashboard → création utilisateur
- Utiliser les fixtures pour l'authentification
- Tester les guards Angular
- Gérer les routes dynamiques (`/users/:id`)
- Utiliser le Trace Viewer pour analyser un test qui échoue

Durée estimée : 45 minutes

Référence : `exercices-jour2.md` → Exercice 4

| 11 · CI/CD & Bonnes pratiques

Intégrer Playwright dans votre pipeline



GitHub Actions — Configuration

```
# .github/workflows/playwright.yml
name: Playwright Tests
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - run: npx playwright install --with-deps
      - run: npx playwright test
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: playwright-report
          path: playwright-report/
```


Parallélisation des tests

```
// playwright.config.ts
export default defineConfig({
  workers: process.env.CI ? 2 : '50%',
  fullyParallel: true,

  // Les tests dans un même fichier sont séquentiels par défaut
  // Pour les paralléliser aussi :
});

// Dans un spec : activer la parallélisation intra-fichier
test.describe.configure({ mode: 'parallel' });
test.describe('Parallel tests', () => {
  test('test 1', async ({ page }) => { });
  test('test 2', async ({ page }) => { });
});
```



Retries & Stabilité

```
// playwright.config.ts
export default defineConfig({
  retries: process.env.CI ? 2 : 0,
  // 0 en développement = échoue immédiatement
  // 2 en CI = réessaie 2 fois avant de marquer échoué
});

// Retry sur un test spécifique
test('flaky test', { retries: 3 }, async ({ page }) => {
  // Ce test sera réessayé jusqu'à 3 fois
});

// Identifier les tests flaky
// npx playwright test --repeat-each=5
```

Bonnes pratiques — Locators

-  Préférer `getByRole`, `getByLabel`, `getByText`
-  Utiliser `data-testid` en dernier recours
-  Éviter les sélecteurs CSS basés sur les classes visuelles
-  Éviter les sélecteurs XPath complexes
-  Éviter les index (`nth(0)`) sauf si indispensable
-  Donner des noms ARIA aux éléments interactifs

Bonnes pratiques — Tests

- ☒ Un test = un scénario utilisateur = un flux métier
- ☒ Utiliser le POM pour tous les sélecteurs répétés
- ☒ Utiliser les fixtures pour le setup (authentification)
- ☒ Éviter les `page.waitForTimeout(1000)` — utiliser l'auto-waiting
- ☒ Activer `trace: 'on-first-retry'` pour le debug en CI
- ☒ Grouper les tests par feature avec `test.describe`



Rapport HTML — Analyser les résultats

```
# Générer et ouvrir le rapport HTML
npx playwright show-report

# Rapport disponible dans playwright-report/index.html
# Contient :
# • Vue d'ensemble : passed / failed / flaky
# • Détail par test avec durée
# • Screenshots des étapes
# • Vidéos d'exécution (si configuré)
# • Trace pour chaque test échoué
```

| 12 · Migration depuis Protractor

Passer à Playwright rapidement



Migration Protractor → Playwright

Concept Protractor	Équivalent Playwright
<code>browser.get('/login')</code>	<code>page.goto('/login')</code>
<code>element(by.css('.btn'))</code>	<code>page.locator('.btn')</code>
<code>element(by.id('x'))</code>	<code>page.locator('#x')</code>
<code>element(by.buttonText('OK'))</code>	<code>page.getByRole('button', {name:'OK'})</code>
<code>browser.wait(EC.visibilityOf(el))</code>	<code>await expect(el).toBeVisible()</code>
<code>browser.sleep(1000)</code>	✗ À supprimer — utiliser l'auto-waiting

Migration — Assertions

Protractor	Playwright
<code>expect(el.getText()).toBe('x')</code>	<code>await expect(el).toHaveText('x')</code>
<code>expect(el.isDisplayed()).toBeTruthy()</code>	<code>await expect(el).toBeVisible()</code>
<code>expect(el.isEnabled()).toBeTruthy()</code>	<code>await expect(el).toBeEnabled()</code>
<code>expect(browser.getCurrentUrl()).toContain('/home')</code>	<code>await expect(page).toHaveURL(/home/)</code>
<code>expect(el.getAttribute('class')).toContain('active')</code>	<code>await expect(el).toHaveClass(/active/)</code>



Migration — Stratégie recommandée

- 1
Installer Playwright en parallèle de Protractor
- 2
Commencer par les nouveaux tests — écrire en Playwright uniquement
- 3
Migrer les smoke tests en premier (les plus critiques)
- 4
Créer les Page Objects au fur et à mesure de la migration
- 5
Supprimer Protractor quand 100% migré



Récapitulatif Jour 2

- ✓ Architecture Playwright (Browser, Context, Page)
- ✓ Installation & playwright.config.ts
- ✓ Locators : getByRole, getByLabel...
- ✓ Auto-waiting et assertions
- ✓ Actions : click, fill, select, drag
- ✓ Interception réseau (page.route)
- ✓ Page Object Model complet
- ✓ Fixtures pour le setup partagé
- ✓ UI Mode et Trace Viewer
- ✓ Playwright Inspector (codegen)
- ✓ CI/CD avec GitHub Actions
- ✓ Migration depuis Protractor

Ce que vous savez maintenant

- Concevoir une **stratégie de tests** complète pour une app Angular
- Écrire des **tests unitaires** Jasmine robustes avec mocks et spies
- Mettre en place **Karma** et comprendre son architecture
- Créer des **tests e2e** Playwright avec les meilleures pratiques
- Organiser votre code avec le **Page Object Model**
- **Débugger** efficacement avec UI Mode et Trace Viewer
- Intégrer les tests dans un **pipeline CI/CD**



Formation terminée !

Merci pour votre participation — Bonne pratique à tous !

*Questions ? Retrouvez le mémento et les exercices dans les
fichiers fournis*

Formation Tests JavaScript